DAA – TOPIC 12

1. Coin Change – Supermarket Billing

• **Aim:** To solve the given problem using a greedy algorithm and obtain the required optimal result.

• **Algorithm:**

1. Identify the available choices and the required objective.
2. Apply the greedy selection rule for the problem.
3. Update the remaining capacity, amount, slots, or tree.
4. Repeat until the solution is complete.
5. Display the final result.

PROGRAM

```
Python Program

def coin_change(coins, amount):
    coins = sorted(coins, reverse=True)
    result = []
    for coin in coins:
        while amount >= coin:
            amount -= coin
            result.append(coin)
    return result

coins = [1, 2, 5, 10]
amount = 28
ans = coin_change(coins, amount)
print(*ans)
print("Minimum coins =", len(ans))
```

OUTPUT

```
Output

10 10 5 2 1
Minimum coins = 5
```

• **Conclusion:** Use the largest denomination first, subtract it from the remaining amount, and continue until the amount becomes zero.

2. Coin Change – Parking Meter

• **Aim:** To solve the given problem using a greedy algorithm and obtain the required optimal result.

• **Algorithm:**

1. Identify the available choices and the required objective.
2. Apply the greedy selection rule for the problem.
3. Update the remaining capacity, amount, slots, or tree.
4. Repeat until the solution is complete.
5. Display the final result.

PROGRAM

```
Python Program

def coin_change(coins, amount):
    coins = sorted(coins, reverse=True)
    result = []
    for coin in coins:
        while amount >= coin:
            amount -= coin
            result.append(coin)
    return result

coins = [1, 5, 10, 25]
amount = 63
ans = coin_change(coins, amount)
print(*ans)
print("Minimum coins =", len(ans))
```

OUTPUT

```
Output

25 25 10 1 1 1
Minimum coins = 6
```

• **Conclusion:** Sort/select coins in descending order and repeatedly choose the largest coin not exceeding the remaining amount.

3. Coin Change – Ticket Counter

• **Aim:** To solve the given problem using a greedy algorithm and obtain the required optimal result.

• **Algorithm:**

1. Identify the available choices and the required objective.
2. Apply the greedy selection rule for the problem.
3. Update the remaining capacity, amount, slots, or tree.
4. Repeat until the solution is complete.
5. Display the final result.

PROGRAM

```
Python Program

def coin_change(coins, amount):
    coins = sorted(coins, reverse=True)
    result = []
    for coin in coins:
        while amount >= coin:
            amount -= coin
            result.append(coin)
    return result

coins = [1, 3, 7, 10]
amount = 24
ans = coin_change(coins, amount)
print(*ans)
print("Minimum coins =", len(ans))
```

OUTPUT

```
Output

10 10 3 1
Minimum coins = 4
```

• **Conclusion:** Repeatedly choose the largest available denomination and reduce the remaining change.

4. Fractional Knapsack – Food Delivery Truck

• **Aim:** To solve the given problem using a greedy algorithm and obtain the required optimal result.

• **Algorithm:**

1. Identify the available choices and the required objective.
2. Apply the greedy selection rule for the problem.
3. Update the remaining capacity, amount, slots, or tree.
4. Repeat until the solution is complete.
5. Display the final result.

PROGRAM

Python Program

```
def fractional_knapsack(weights, profits, capacity):
    items = sorted(
        zip(weights, profits),
        key=lambda x: x[1] / x[0],
        reverse=True
    )
    total = 0
    for weight, profit in items:
        if capacity == 0:
            break
        take = min(weight, capacity)
        total += take * profit / weight
        capacity -= take
    return total

weights = [10, 20, 30]
profits = [60, 100, 120]
capacity = 50
print("Maximum profit =", int(fractional_knapsack(
    weights, profits, capacity)))
```

OUTPUT

Output

```
Maximum profit = 240
```

• **Conclusion:** Compute profit/weight ratios, sort items by ratio, take complete items when possible, then take a fraction of the next item.

5. Fractional Knapsack – Gold Transport

• **Aim:** To solve the given problem using a greedy algorithm and obtain the required optimal result.

• **Algorithm:**

1. Identify the available choices and the required objective.
2. Apply the greedy selection rule for the problem.
3. Update the remaining capacity, amount, slots, or tree.
4. Repeat until the solution is complete.
5. Display the final result.

PROGRAM

Python Program

```
def fractional_knapsack(weights, profits, capacity):
    items = sorted(
        zip(weights, profits),
        key=lambda x: x[1] / x[0],
        reverse=True
    )
    total = 0
    for weight, profit in items:
        if capacity == 0:
            break
        take = min(weight, capacity)
        total += take * profit / weight
        capacity -= take
    return total

weights = [5, 10, 15]
profits = [30, 40, 45]
capacity = 20
print("Maximum profit =", int(fractional_knapsack(
    weights, profits, capacity)))
```

OUTPUT

Output

```
Maximum profit = 80
```

• **Conclusion:** Calculate value density for every item, sort by density, and fill the capacity greedily, allowing fractional items.

6. Fractional Knapsack – Relief Material Loading

• **Aim:** To solve the given problem using a greedy algorithm and obtain the required optimal result.

• **Algorithm:**

1. Identify the available choices and the required objective.
2. Apply the greedy selection rule for the problem.
3. Update the remaining capacity, amount, slots, or tree.
4. Repeat until the solution is complete.
5. Display the final result.

PROGRAM

Python Program

```
def fractional_knapsack(weights, profits, capacity):
    items = sorted(
        zip(weights, profits),
        key=lambda x: x[1] / x[0],
        reverse=True
    )
    total = 0
    for weight, profit in items:
        if capacity == 0:
            break
        take = min(weight, capacity)
        total += take * profit / weight
        capacity -= take
    return total

weights = [4, 8, 2, 6]
profits = [20, 40, 14, 30]
capacity = 10
print("Maximum profit =", int(fractional_knapsack(
    weights, profits, capacity)))
```

OUTPUT

Output

```
Maximum profit = 54
```

• **Conclusion:** Sort relief items by profit-to-weight ratio and select the highest ratio items until the truck capacity is full.

7. Job Sequencing – Software Company Tasks

• **Aim:** To solve the given problem using a greedy algorithm and obtain the required optimal result.

• **Algorithm:**

1. Identify the available choices and the required objective.
2. Apply the greedy selection rule for the problem.
3. Update the remaining capacity, amount, slots, or tree.
4. Repeat until the solution is complete.
5. Display the final result.

PROGRAM

```
Python Program

def job_sequencing(jobs, deadlines, profits):
    data = sorted(zip(jobs, deadlines, profits),
                  key=lambda x: x[2], reverse=True)
    slots = [None] * (max(deadlines) + 1)
    total = 0
    for job, deadline, profit in data:
        for s in range(deadline, 0, -1):
            if slots[s] is None:
                slots[s] = job
                total += profit
                break
    return [x for x in slots[1:] if x], total

jobs = A B C D
deadlines = [2, 1, 2, 1]
profits = [100, 19, 27, 25]
selected, total = job_sequencing(jobs, deadlines, profits)
print("Selected jobs =", *selected)
print("Maximum profit =", total)
```

OUTPUT

```
Output

Selected jobs = C A
Maximum profit = 127
```

• **Conclusion:** Sort jobs by decreasing profit and place each job in the latest free slot before its deadline.

8. Job Sequencing – Exam Paper Evaluation

• **Aim:** To solve the given problem using a greedy algorithm and obtain the required optimal result.

• **Algorithm:**

1. Identify the available choices and the required objective.
2. Apply the greedy selection rule for the problem.
3. Update the remaining capacity, amount, slots, or tree.
4. Repeat until the solution is complete.
5. Display the final result.

PROGRAM

```
Python Program

def job_sequencing(jobs, deadlines, profits):
    data = sorted(zip(jobs, deadlines, profits),
                  key=lambda x: x[2], reverse=True)
    slots = [None] * (max(deadlines) + 1)
    total = 0
    for job, deadline, profit in data:
        for s in range(deadline, 0, -1):
            if slots[s] is None:
                slots[s] = job
                total += profit
                break
    return [x for x in slots[1:] if x], total

jobs = J1 J2 J3 J4 J5
deadlines = [2, 1, 2, 1, 3]
profits = [20, 15, 10, 5, 1]
selected, total = job_sequencing(jobs, deadlines, profits)
print("Selected jobs =", *selected)
print("Maximum profit =", total)
```

OUTPUT

```
Output

Selected jobs = J2 J1 J5
Maximum profit = 36
```

• **Conclusion:** Sort jobs by profit, then assign each job to the latest available slot not exceeding its deadline.

9. Job Sequencing – Printing Press Orders

• **Aim:** To solve the given problem using a greedy algorithm and obtain the required optimal result.

• **Algorithm:**

1. Identify the available choices and the required objective.
2. Apply the greedy selection rule for the problem.
3. Update the remaining capacity, amount, slots, or tree.
4. Repeat until the solution is complete.
5. Display the final result.

PROGRAM

```
Python Program

def job_sequencing(jobs, deadlines, profits):
    data = sorted(zip(jobs, deadlines, profits),
                  key=lambda x: x[2], reverse=True)
    slots = [None] * (max(deadlines) + 1)
    total = 0
    for job, deadline, profit in data:
        for s in range(deadline, 0, -1):
            if slots[s] is None:
                slots[s] = job
                total += profit
                break
    return [x for x in slots[1:] if x], total

jobs = A B C D E
deadlines = [2, 1, 2, 1, 3]
profits = [100, 50, 10, 20, 30]
selected, total = job_sequencing(jobs, deadlines, profits)
print("Selected jobs =", *selected)
print("Maximum profit =", total)
```

OUTPUT

```
Output

Selected jobs = B A E
Maximum profit = 180
```

• **Conclusion:** Arrange jobs in decreasing profit order and place them in the latest free slot within their deadlines.

10. Job Sequencing – Freelance Projects

• **Aim:** To solve the given problem using a greedy algorithm and obtain the required optimal result.

• **Algorithm:**

1. Identify the available choices and the required objective.
2. Apply the greedy selection rule for the problem.
3. Update the remaining capacity, amount, slots, or tree.
4. Repeat until the solution is complete.
5. Display the final result.

PROGRAM

```
Python Program

def job_sequencing(jobs, deadlines, profits):
    data = sorted(zip(jobs, deadlines, profits),
                  key=lambda x: x[2], reverse=True)
    slots = [None] * (max(deadlines) + 1)
    total = 0
    for job, deadline, profit in data:
        for s in range(deadline, 0, -1):
            if slots[s] is None:
                slots[s] = job
                total += profit
                break
    return [x for x in slots[1:] if x], total

jobs = P1 P2 P3 P4
deadlines = [1, 1, 2, 2]
profits = [40, 30, 20, 10]
selected, total = job_sequencing(jobs, deadlines, profits)
print("Selected jobs =", *selected)
print("Maximum profit =", total)
```

OUTPUT

```
Output

Selected jobs = P1 P3
Maximum profit = 60
```

• **Conclusion:** Choose projects by descending profit and schedule each in the latest available slot before its deadline.

11. Huffman Coding – Text Compression

• **Aim:** To solve the given problem using a greedy algorithm and obtain the required optimal result.

• **Algorithm:**

1. Identify the available choices and the required objective.
2. Apply the greedy selection rule for the problem.
3. Update the remaining capacity, amount, slots, or tree.
4. Repeat until the solution is complete.
5. Display the final result.

PROGRAM

Python Program

```
import heapq

def huffman(chars, freq):
    heap = [[f, c] for c, f in zip(chars, freq)]
    heapq.heapify(heap)
    while len(heap) > 1:
        f1, l = heapq.heappop(heap)
        f2, r = heapq.heappop(heap)
        heapq.heappush(heap, [f1 + f2, [l, r]])
    codes = {}
    def walk(node, code=""):
        if isinstance(node, str):
            codes[node] = code or "0"
            return
        walk(node[0], code + "0")
        walk(node[1], code + "1")
    walk(heap[0][1])
    return codes

chars = "a b c d e f".split()
freq = [5, 9, 12, 13, 16, 45]
for c, code in huffman(chars, freq).items():
    print(f"{c}: {code}")
```

OUTPUT

Output

```
f: 0
c: 100
d: 101
a: 1100
b: 1101
e: 111
```

• **Conclusion:** Put symbols in a min-priority queue, repeatedly merge the two least frequent nodes, and assign 0/1 along the resulting Huffman tree.

12. Huffman Coding – Message Encoding

• **Aim:** To solve the given problem using a greedy algorithm and obtain the required optimal result.

• **Algorithm:**

1. Identify the available choices and the required objective.
2. Apply the greedy selection rule for the problem.
3. Update the remaining capacity, amount, slots, or tree.
4. Repeat until the solution is complete.
5. Display the final result.

PROGRAM

```
Python Program
import heapq

def huffman(chars, freq):
    heap = [[f, c] for c, f in zip(chars, freq)]
    heapq.heapify(heap)
    while len(heap) > 1:
        f1, l = heapq.heappop(heap)
        f2, r = heapq.heappop(heap)
        heapq.heappush(heap, [f1 + f2, [l, r]])
    codes = {}
    def walk(node, code=""):
        if isinstance(node, str):
            codes[node] = code or "0"
            return
        walk(node[0], code + "0")
        walk(node[1], code + "1")
    walk(heap[0][1])
    return codes

chars = "A B C D".split()
freq = [10, 20, 30, 40]
for c, code in huffman(chars, freq).items():
    print(f"{c}: {code}")
```

OUTPUT

```
Output
D: 0
C: 10
A: 110
B: 111
```

• **Conclusion:** Build a Huffman tree by repeatedly merging the two smallest frequencies, then traverse the tree to generate binary codes.

13. Huffman Coding – Sensor Data Compression

• **Aim:** To solve the given problem using a greedy algorithm and obtain the required optimal result.

• **Algorithm:**

1. Identify the available choices and the required objective.
2. Apply the greedy selection rule for the problem.
3. Update the remaining capacity, amount, slots, or tree.
4. Repeat until the solution is complete.
5. Display the final result.

PROGRAM

Python Program

```
import heapq

def huffman(chars, freq):
    heap = [[f, c] for c, f in zip(chars, freq)]
    heapq.heapify(heap)
    while len(heap) > 1:
        f1, l = heapq.heappop(heap)
        f2, r = heapq.heappop(heap)
        heapq.heappush(heap, [f1 + f2, [l, r]])
    codes = {}
    def walk(node, code=""):
        if isinstance(node, str):
            codes[node] = code or "0"
            return
        walk(node[0], code + "0")
        walk(node[1], code + "1")
    walk(heap[0][1])
    return codes

chars = "x y z w".split()
freq = [7, 8, 14, 25]
for c, code in huffman(chars, freq).items():
    print(f"{c}: {code}")
```

OUTPUT

Output

```
w: 0
z: 10
x: 110
y: 111
```

• **Conclusion:** Construct the Huffman tree from the lowest-frequency symbols first and assign shorter codes to more frequent symbols.

14. Coin Change – Vending Machine

• **Aim:** To solve the given problem using a greedy algorithm and obtain the required optimal result.

• **Algorithm:**

1. Identify the available choices and the required objective.
2. Apply the greedy selection rule for the problem.
3. Update the remaining capacity, amount, slots, or tree.
4. Repeat until the solution is complete.
5. Display the final result.

PROGRAM

```
Python Program

def coin_change(coins, amount):
    coins = sorted(coins, reverse=True)
    result = []
    for coin in coins:
        while amount >= coin:
            amount -= coin
            result.append(coin)
    return result

coins = [1, 2, 5]
amount = 11
ans = coin_change(coins, amount)
print(*ans)
print("Minimum coins =", len(ans))
```

OUTPUT

```
Output

5 5 1
Minimum coins = 3
```

• **Conclusion:** Select the largest usable denomination repeatedly until the vending-machine change amount is reached.

15. Fractional Knapsack – Warehouse Packing

• **Aim:** To solve the given problem using a greedy algorithm and obtain the required optimal result.

• **Algorithm:**

1. Identify the available choices and the required objective.
2. Apply the greedy selection rule for the problem.
3. Update the remaining capacity, amount, slots, or tree.
4. Repeat until the solution is complete.
5. Display the final result.

PROGRAM

Python Program

```
def fractional_knapsack(weights, profits, capacity):
    items = sorted(
        zip(weights, profits),
        key=lambda x: x[1] / x[0],
        reverse=True
    )
    total = 0
    for weight, profit in items:
        if capacity == 0:
            break
        take = min(weight, capacity)
        total += take * profit / weight
        capacity -= take
    return total

weights = [2, 3, 5, 7]
profits = [10, 5, 15, 7]
capacity = 8
print("Maximum profit =", int(fractional_knapsack(
    weights, profits, capacity)))
```

OUTPUT

Output

```
Maximum profit = 30
```

• **Conclusion:** Calculate profit/weight ratios, sort in descending order, take whole items while possible, and take a fraction when necessary.